Tu responsabilidad y mapa de archivos

Archivo: src/pipeline.py | Función o clase: ejecutar_lote | Inicio: línea 101

Tu parte convierte fuentes distintas a un esquema común y conecta el ETL. Maki aporta calidad y limpieza; Leo, validación y persistencia. También preparaste el acceso por consola y la interfaz de carga y consulta. Los roles no atribuyen cada línea exclusivamente a una persona.

Mapa: src/lectura.py lee; src/catalogo.py activa cuentas; src/integracion.py normaliza y cruza; src/transformacion.py escala y segmenta; src/pipeline.py coordina; src/cli.py ofrece comandos; src/api.py recibe archivos; src/consultas.py prepara resultados.

ejecutar_lote recibe ruta, fuente, catálogo opcional y ruta de base. LOCK evita ejecutar dos cargas simultáneas en este proceso. Se lee el archivo, se obtiene la versión del catálogo y se calcula la identidad antes de llamar a _run.

Fragmento real, líneas 101-107. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
101 def ejecutar_lote(ruta,fuente,ruta_catalogo=None,ruta_bd='outputs/pipeline.sqlite'):
102     with LOCK:
103         content=leer_fuente(ruta,fuente)
104         if ruta_catalogo:activar_catalogo(ruta_catalogo,ruta_bd)
105         version,_=obtener_catalogo(ruta_bd)
106         identity=build_load_identity(fuente,{'archivo':content['metadatos']['hash']},version
    ,RULES,PROCESS)
107         return _run(content['registros'],fuente,ruta_bd,{**content['metadatos'],'file_hashes
    ':{'archivo':content['metadatos']['hash']}},identity)
```

Lectura: formatos, columnas y origen

Archivo: src/lectura.py | Función o clase: leer_fuente | Inicio: línea 16

FIELDS define las columnas por fuente. DIGITAL usa JSON con lista de objetos. ATM admite CSV o XLSX con hoja transacciones_atm. El catálogo de la interfaz es CSV. Los nombres de archivo son libres; la estructura no lo es.

El lector usa UTF-8 con BOM admitido y comprueba encabezados únicos y completos en CSV/XLSX. Una fila con longitud incorrecta queda marcada _fila_invalida. La lectura no limpia ni inventa valores. Un archivo estructuralmente ilegible se rechaza completo.

La salida contiene registros y metadatos: archivo, hash SHA-256, fuente y offset. offset permite conservar números de fila considerando los encabezados. _run añade original_json y fila_origen para rastrear cada resultado.

Fragmento real, líneas 16-30. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
16 def leer_fuente(ruta, fuente):
17     ruta = Path(ruta)
18     if fuente not in FIELDS:
19         raise ValueError('Fuente desconocida')
20     raw = ruta.read_bytes()
21     ext = ruta.suffix.lower()
22     if fuente == 'DIGITAL' and ext == '.json':
23         rows = json.loads(raw.decode('utf-8-sig'), parse_constant=lambda value: (_ for _ in
   ()).throw(ValueError('Número JSON no finito: '+value)))
24         if not isinstance(rows, list):
25             raise ValueError('El JSON debe contener una lista')
26     elif fuente in ('ATM', 'CATALOGO') and ext in ('.csv', '.xlsx'):
27         if ext == '.csv':
28             matrix = list(csv.reader(raw.decode('utf-8-sig').splitlines()))
29         else:
30             try:
```

Normalización y conversión de moneda

Archivo: src/integracion.py | Función o clase: normalizar | Inicio: línea 19

normalizar llama a validate_atm_record_detailed o validate_digital_record_detailed. Si hay errores, guarda un rechazo; si no, obtiene model_dump y construye nombres comunes: id_transaccion, id_cuenta, fecha_hora, estado_operacion y tipo_operacion.

TASA es Decimal(19.50). money usa quantize y ROUND_HALF_UP para centavos. Un monto USD se multiplica por 19.50; MXN usa factor 1. Se conservan moneda_original, monto_original, tasa_conversion y original_json. Un monto faltante sigue faltante para la etapa de imputación.

Ejemplo SPEI-2026-00003: 28312.37 USD por 19.50 = 552091.215; se guarda 552091.22 MXN. IQR produce después otro campo analítico. No se sustituye el importe real por el recortado. Fechas: validadores interpretan la zona y fecha deriva YYYY-MM-DD.

Fragmento real, líneas 44-57. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
44              'fecha_hora': when, 'fecha': when.date().isoformat(), 'estado_operacion':
   status,
45              'tipo_operacion': values.get('tipo_operacion', 'TRANSFERENCIA'),
   'moneda_original': currency,
46              'monto_original': amount, 'monto_mxn': money(amount * (TASA if currency ==
   'USD' else 1)) if amount is not None else None,
47              'tasa_conversion': TASA if currency == 'USD' else Decimal(1),
   'registro_origen': origin}
48          if fuente == 'ATM' and r.get('comision_aplicada') is not None:
49              r['comision_aplicada'] = money(r['comision_aplicada'])
50          changes = []
51          for key, value in values.items():
52              before = original.get(key)
53              if str(before) != str(value):
54                  changes.append({'campo': key, 'antes': before, 'despues': value, 'regla':
   'NORMALIZACION'})
55          if currency == 'USD':
56              changes.append({'campo': 'monto_mxn', 'antes': amount, 'despues':
   r['monto_mxn'], 'regla': 'USD_MXN_19.50'})
57          r['transformaciones'] = changes
```

Catálogo y cuentas huérfanas

Archivo: src/integracion.py | Función o clase: cruzar_catalogo | Inicio: línea 63

cruzar_catalogo recibe candidatos y un diccionario de cuentas. Busca id_cuenta con get. Si no existe, genera CUENTA_NO_REGISTRADA; si existe, añade cuenta y estado_cuenta. Un cruce no multiplica filas.

src/catalogo.py: activar_catalogo valida claves, titulares, fechas, límites y estados, crea una versión por hash y la activa en una transacción. Un catálogo nuevo no puede omitir cuentas anteriores. obtener_catalogo devuelve versión y cuentas vigentes.

ATM-TX-00009 usa CTA-8997, ausente del catálogo, y va a cuarentena. Una cuenta BLOQUEADA existente puede conservarse porque no conocemos su estado histórico al ocurrir la operación. El pipeline evalúa datos; no autoriza retiros.

Fragmento real, líneas 63-70. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
63 def cruzar_catalogo(registros, catalogo):
64     candidates, rejected = [], []
65     for r in registros:
66         account = catalogo.get(r['id_cuenta'])
67         if account is None:
68             rejected.append(rechazo(r, 'CUENTA_NO_REGISTRADA', 'id_cuenta')); continue
69         candidates.append({**r, 'cuenta': account, 'estado_cuenta':
   account['estado_cuenta']})
70     return {'candidatos': candidates, 'rechazados': rejected}
```

Min-Max y segmentos

Archivo: src/transformacion.py | Función o clase: escalar_segmentar | Inicio: línea 19

Entrada: candidatos después de IQR. Se agrupan por fuente y operación. Se calculan mínimo, máximo y percentiles 1/3 y 2/3 con percentil. Normalizado = (valor - mínimo)/(máximo - mínimo). Si el grupo es constante, el resultado es cero.

La variable usada es monto_analitico_mxn. BAJO llega hasta el primer corte; MEDIO hasta el segundo; ALTO lo supera. Las consultas de saldo quedan en cero y BAJO. AnaliticaFinal valida importes Decimal finitos y escala float entre cero y uno.

ATM-TX-00034: (4000 - 500)/(12000 - 500) = 0.3043478261, segmento MEDIO. El 0.3043 no expresa probabilidad de fraude. Los parámetros se guardan por lote; no compares escalas de lotes distintos sin revisarlos.

Fragmento real, líneas 37-46. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
37         if lo == hi and reference and value != lo:
38             rejected.append({'original_json': r, 'motivo':
   'REFERENCIA_ESTADISTICA_INSUFICIENTE'}); continue
39         scaled = (value-lo)/(hi-lo) if hi != lo else Decimal(0)
40         r['fuera_rango_referencia'] = scaled < 0 or scaled > 1
41         r['monto_normalizado'] = float(min(1,max(0,scaled)))
42         r['segmento'] = 'BAJO' if value <= a else 'MEDIO' if value <= b else 'ALTO'
43         r['rango_constante'] = lo == hi
44         AnaliticaFinal.model_validate({k:r[k] for k in AnaliticaFinal.model_fields})
45         candidates.append(r)
46     return {'candidatos':candidates,'rechazados':rejected,'parametros':params}
```

Orquestación y conciliación

Archivo: src/pipeline.py | Función o clase: _run | Inicio: línea 22

_run perfila antes de limpiar, separa copias y normaliza. Agrupa por identidad; si las huellas difieren, rechaza el conflicto. Consulta las claves existentes: una operación idéntica ya cargada se omite; una distinta con la misma clave se rechaza.

Después cruza cuentas, imputa, aplica IQR y escala. Acumula todos los rechazos y construye perfil_final, comparacion, parametros y etapas. La igualdad final exige leídas = aceptadas + cuarentena + omitidas.

persist_batch guarda las salidas y la evidencia. Las siete etapas de la página describen poblaciones diferentes: no se suman. reprocesar permite volver a evaluar un rechazo desde la consola conservando el historial.

Fragmento real, líneas 52-67. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
52     joined=cruzar_catalogo(new,catalog);bad.extend(joined['rechazados']);eligible=joined['ca
   ndidatos']
53     filled=imputar(eligible)
54     bad.extend(_error(r) for r in filled['rechazados'])
55     for r in filled['candidatos']:
56         r['monto_mxn']=money(r['monto_mxn']);r['version_catalogo']=version
57         r['transformaciones'].extend(r.get('_cambios_imputacion',[]))
58         if source=='ATM':r['comision_aplicada']=money(r['comision_aplicada'])
59     capped=aplicar_iqr(filled['candidatos'])
60     for r in capped['candidatos']:
61         r['monto_analitico_mxn']=money(r['monto_analitico_mxn'])
62     scaled=escalar_segmentar(capped['candidatos'])
63     bad.extend(_error(r) for r in scaled['rechazados'])
64     for r in scaled['candidatos']:
65         r['monto_analitico_mxn']=money(r['monto_analitico_mxn'])
66         r['referencia_ejecucion']=metadata.get('referencia_ejecucion')
67     metadata['parametros']={'imputacion':filled['parametros'],'iqr':capped['parametros'],'es
   cala':scaled['parametros']}
```

Carga de archivos y página actual

Archivo: src/api.py | Función o clase: preview | Inicio: línea 85

create_app construye FastAPI. save_upload comprueba fuente, extensión, tamaño máximo de 20 MB y nombre seguro. preview lee una muestra de cinco filas sin ingerirla y elimina su archivo temporal al terminar. upload activa el catálogo o llama a ejecutar_lote.

Rutas: POST /api/archivos/vista-previa muestra la entrada; POST /api/cargas procesa; POST /api/sesiones crea una base vacía; GET /api/control consulta resultados; GET /api/estado devuelve catálogo y esquemas. Una sesión nueva conserva las bases anteriores.

La página no está escrita solo en Python: web/index.html contiene la estructura; web/base.css el estilo; web/common.js la navegación; web/workflow.js la cola de archivos; web/dashboard.js las gráficas y tablas. La cola procesa primero catálogo y luego lotes. Es apoyo para mostrar el ETL, no una fase exigida por la rúbrica.

Fragmento real, líneas 85-99. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
85      def preview(fuente: str = Form(...), archivo: UploadFile = File(...)):
86          with LOCK:
87              path = save_upload(fuente, archivo)
88              try:
89                  data = leer_fuente(path, fuente)
90                  rows = data['registros']
91                  if not rows:
92                      raise ValueError('El archivo no contiene registros.')
93                  result = {'archivo': path.name, 'filas': len(rows),
94                            'muestra': rows[:5], 'campos': FIELDS[fuente]}
95                  return json.loads(json.dumps(result, default=str))
96              except (OSError, UnicodeError, json.JSONDecodeError) as exc:
97                  raise ValueError('No se pudo leer el archivo: ' + str(exc)) from exc
98              finally:
99                  path.unlink(missing_ok=True)
```

Consulta de resultados y ejecución

Archivo: src/consultas.py | Función o clase: snapshot | Inicio: línea 17

snapshot lee transacciones_limpias, cuarentena_errores, ejecuciones, evidencia_lotes y filas_omitidas. Convierte payload JSON a campos para las tablas. matches aplica filtros. Counter cuenta motivos; una fila puede tener varios, por eso sus sumas no equivalen al total de cuarentena.

Desde la raíz: .venv/Scripts/python.exe -m uvicorn src.api:app --host 127.0.0.1 --port 8765. Abre http://127.0.0.1:8765. Mantén la terminal abierta; Ctrl+C detiene el servidor. Ya no existe un lanzador .cmd.

Para practicar divisas y escala: .venv/Scripts/python.exe docs/guias/consultar_ejemplos.py avilix. Para consola, main en src/cli.py implementa cargar, resumen, actualizar-catalogo y reprocesar. Los comandos completos están en README.md.

Fragmento real, líneas 17-28. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
17 def snapshot(db,filters=None):
18     f=filters or {};c=initialize_database(db)
19     try:
20         clean=[]
21         for row in c.execute('SELECT * FROM transacciones_limpias'):
22             r={**json.loads(row['payload']),**{k:row[k] for k in
   ('id','ejecucion_id','fecha_procesado')}}
23             if matches(r,f):clean.append(r)
24         quarantine=[]
25         for row in c.execute('SELECT * FROM cuarentena_errores'):
26             r=dict(row);original=json.loads(r['original_record']);origin=original.get('origi
   nal_json',original)
27             if not isinstance(origin,dict):origin={}
28             r.update(id_cuenta=original.get('id_cuenta',origin.get('account_id','')),tipo_op
   eracion=original.get('tipo_operacion','TRANSFERENCIA' if r['fuente']=='DIGITAL' else ''),est
   ado_operacion=original.get('estado_operacion',origin.get('resultado',origin.get('status',''))
   )),origen_ingreso=original.get('origen_ingreso','ARCHIVO'),fecha_hora=original.get('fecha_ho
   ra',origin.get('timestamp',''))))
```

Repaso y archivos para entregar

ETL significa extraer, transformar y cargar. Puedes explicar tu módulo indicando su entrada, la decisión que toma, su salida y la evidencia que conserva. No basta con leer nombres de técnicas.

Los ejemplos de estas guías corresponden a datasets/catalogo_cuentas.csv, datasets/transacciones_atm.xlsx y datasets/transferencias_digitales.json. Con archivos nuevos cambian las estadísticas y resultados. La tasa académica USD/MXN es fija: 19.50.

Entrega al profesor: enlace al repositorio Git, entrega/pipeline_banco.sqlite y entrega/REPORTE_FINAL.pdf. Dentro del repositorio quedan reglas, diagrama, código y pruebas de resiliencia. Las guías de docs/guias son material de estudio del equipo.

El video y la presentación indicados posteriormente por el profesor se preparan aparte; esta guía no equivale a haber grabado o expuesto. docs/GUION_VIDEO.md propone el recorrido. La interfaz es un apoyo visual para demostrar el pipeline.

Para localizar el código, abre el archivo .py en el editor y busca el nombre de función indicado. Las líneas corresponden a esta versión y pueden moverse con futuros cambios. Los fragmentos se extraen del código al regenerar las guías; no son pseudocódigo.

Referencias: guía del profesor, puntos 4 y 6; REGLAS_DE_NEGOCIO.md; docs/CONTRATO_DATOS.md; código y pruebas citados; docs/evidencia/resultados.json; base SQLite de entrega. Regla vigente 1.5, proceso 2.1.